CSE 312: Database Management System Lab

Experiment No. 06: Normalization

1. Objective

The objectives of this experiment are:

- To understand the concept of normalization and why it is needed in relational database design.
- To learn about functional dependencies and how they drive the normalization process.
- To apply First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) step by step.
- To practice decomposing unnormalized tables into well-structured relations using functional dependencies.

2. Introduction

Normalization is a systematic process of organizing data in a relational database to reduce data redundancy and eliminate undesirable anomalies such as insertion, update, and deletion anomalies. It was first proposed by Edgar F. Codd in 1970.

When a database is poorly designed, the same piece of information may be stored in multiple places. This leads to wasted storage, inconsistency when data is updated in one place but not another, and difficulties in inserting or deleting records. Normalization addresses these problems by decomposing large, flat tables into smaller, well-structured relations based on functional dependencies.

Each level of normalization (called a *normal form*) builds upon the previous one, imposing stricter rules on the structure of the relation. In this experiment, we will cover the first three normal forms: 1NF, 2NF, and 3NF.

3. Key Concepts

3.1 Functional Dependency

A functional dependency (FD) is a relationship between two sets of attributes in a relation. We say that attribute B is functionally dependent on attribute A (written as

$A \to B$) if, for every valid instance of A, there is exactly one value of B associated with it.

Example: In a `Students` table, if `StudentID` uniquely determines `StudentName`, we write:

```
StudentID -> StudentName
```

This means: given a particular `StudentID`, there can be only one corresponding `StudentName`.

3.2 Types of Functional Dependencies

- **Full Functional Dependency:** An attribute B is fully functionally dependent on a composite key $\{A_1, A_2\}$ if B depends on the *entire* key and not on any proper subset of it.
- **Partial Functional Dependency:** An attribute B is partially dependent on a composite key $\{A_1, A_2\}$ if B depends on only a part of the key (e.g., $A_1 \to B$).
- **Transitive Functional Dependency:** If $A \to B$ and $B \to C$, then C is transitively dependent on A through B.

3.3 Candidate Key and Primary Key

A **candidate key** is a minimal set of attributes that can uniquely identify every tuple in a relation. A **primary key** is the candidate key chosen to uniquely identify tuples. All non-key attributes must be functionally determined by the primary key.

3.4 Anomalies in Unnormalized Data

Consider the following unnormalized `StudentCourse` table:

StudentID	StudentName	CourseID	CourseName	Instructor
101	Alice	CS101	DBMS	Dr. Karim
101	Alice	CS102	OS	Dr. Hasan
102	Bob	CS101	DBMS	Dr. Karim
103	Charlie	CS103	Networks	Dr. Rahim

This table suffers from:

- **Insertion Anomaly:** We cannot add a new course unless a student enrolls in it.
- **Update Anomaly:** If Dr. Karim's name changes, we must update every row where CS101 appears.

- **Deletion Anomaly:** If Charlie drops CS103, we lose all information about the Networks course.

Normalization resolves these problems systematically.

4. First Normal Form (1NF)

A relation is in **First Normal Form (1NF)** if and only if:

- All columns contain only atomic (indivisible) values.
- There are no repeating groups or multi-valued attributes.
- Each row is unique (a primary key exists).

4.1 Example: Unnormalized Table (UNF)

Consider the following `StudentCourse` table where a single student row contains multiple courses:

StudentID	StudentName	Courses
101	Alice	CS101, CS102
102	Bob	CS101
103	Charlie	CS103, CS101, CS104

The `Courses` column contains multiple values in a single cell. This violates 1NF.

4.2 Conversion to 1NF

To convert to 1NF, we eliminate multi-valued attributes by creating separate rows for each atomic value:

StudentID	StudentName	CourseID
101	Alice	CS101
101	Alice	CS102
102	Bob	CS101
103	Charlie	CS103
103	Charlie	CS101
103	Charlie	CS104

Now every cell contains a single atomic value.
The composite primary key is `{StudentID, CourseID}`.

Functional Dependencies at this stage:

```
{StudentID, CourseID} -> StudentName    (Primary Key -> Non-key)
StudentID -> StudentName                (Partial Dependency)
```

The table is now in 1NF, but partial dependencies exist. This will be resolved in 2NF.

5. Second Normal Form (2NF)

A relation is in **Second Normal Form (2NF)** if and only if:

- It is already in 1NF.
- Every non-key attribute is fully functionally dependent on the *entire* primary key (i.e., there are no partial dependencies).

2NF is relevant only when the primary key is composite. If the primary key is a single attribute, a 1NF table is automatically in 2NF.

5.1 Identifying Partial Dependencies

Consider the following 1NF table `Enrollment` with composite primary key `{StudentID, CourseID}`:

StudentID	CourseID	StudentName	CourseName	Grade
101	CS101	Alice	DBMS	A
101	CS102	Alice	OS	B+
102	CS101	Bob	DBMS	A-
103	CS103	Charlie	Networks	B

Primary Key: `{StudentID, CourseID}`
Functional Dependencies:

```
{StudentID, CourseID} -> Grade          (Full Dependency)
StudentID -> StudentName                (Partial Dependency)
CourseID -> CourseName                  (Partial Dependency)
```

Here, `StudentName` depends only on `StudentID` (a part of the key), and `CourseName` depends only on `CourseID` (another part of the key). These are partial dependencies.

5.2 Conversion to 2NF

To convert to 2NF, we remove partial dependencies by decomposing the table. Each partial dependency becomes its own table:

Step 1: Create a table for each partial dependency.
Step 2: Keep the fully dependent attributes with the original composite key.

Table 1: Students

StudentID (PK)	StudentName
101	Alice
102	Bob
103	Charlie

Functional Dependency: `StudentID -> StudentName`

Table 2: Courses

CourseID (PK)	CourseName
CS101	DBMS
CS102	OS
CS103	Networks

Functional Dependency: `CourseID -> CourseName`

Table 3: Enrollment

StudentID (PK, FK)	CourseID (PK, FK)	Grade
101	CS101	A
101	CS102	B+
102	CS101	A-
103	CS103	B

Functional Dependency: `{StudentID, CourseID} -> Grade`

All non-key attributes now fully depend on their respective primary keys. The tables are in 2NF. However, transitive dependencies may still exist, which will be resolved in 3NF.

6. Third Normal Form (3NF)

A relation is in **Third Normal Form (3NF)** if and only if:

- It is already in 2NF.
- No non-key attribute is transitively dependent on the primary key (i.e., every non-key attribute depends directly on the primary key and nothing else).

In other words, for every functional dependency $X \to A$ in the relation, at least one of the following must hold:

- X is a superkey, or
- A is part of a candidate key (a prime attribute).

6.1 Identifying Transitive Dependencies

Consider the following 2NF table `StudentDept`:

StudentID	StudentName	DeptID	DeptName
101	Alice	D01	CSE
102	Bob	D02	EEE
103	Charlie	D01	CSE
104	Diana	D03	CE

Primary Key: `StudentID`

Functional Dependencies:

```
StudentID -> StudentName        (Direct Dependency)
StudentID -> DeptID             (Direct Dependency)
DeptID -> DeptName              (Direct Dependency on non-key)
```

Since `StudentID -> DeptID` and `DeptID -> DeptName`, we have:

```
StudentID -> DeptID -> DeptName    (Transitive Dependency)
```

The attribute `DeptName` does not depend directly on the primary key `StudentID`. Instead, it depends on `DeptID`, which is itself a non-key attribute. This is a transitive dependency and violates 3NF.

6.2 Conversion to 3NF

To convert to 3NF, we remove transitive dependencies by decomposing the table. The attribute causing the transitive dependency (`DeptID -> DeptName`) is moved to a new table.

Step 1: Create a new table for the transitive dependency.

Step 2: Remove the transitively dependent attribute from the original table and keep the determinant as a foreign key.

Table 1: Students

StudentID (PK)	StudentName	DeptID (FK)
101	Alice	D01
102	Bob	D02
103	Charlie	D01
104	Diana	D03

Functional Dependencies:

```
StudentID -> StudentName
StudentID -> DeptID
```

Table 2: Departments

DeptID (PK)	DeptName
D01	CSE
D02	EEE
D03	CE

Functional Dependency: `DeptID -> DeptName`

Now every non-key attribute depends directly and only on the primary key in its respective table. There are no transitive dependencies. Both tables are in 3NF.

7. Complete Walkthrough: UNF to 3NF

This section demonstrates the full normalization process on a single table from an unnormalized state all the way to 3NF.

7.1 Original Unnormalized Table

Consider a `Library` table that tracks book borrowing:

MemberID	MemberName	Books	Authors	Branch	BranchCity
M01	Rahim	B01, B02	Tanenbaum, Navathe	Central	Dhaka
M02	Karim	B01	Tanenbaum	Mirpur	Dhaka
M03	Sumon	B03	Silberschatz	Uttara	Dhaka

Problems: Multi-valued columns (`Books`, `Authors`), redundancy, and anomalies.

7.2 Step 1: Convert to 1NF

Eliminate multi-valued attributes by expanding rows:

MemberID	MemberName	BookID	Author	Branch	BranchCity
M01	Rahim	B01	Tanenbaum	Central	Dhaka
M01	Rahim	B02	Navathe	Central	Dhaka
M02	Karim	B01	Tanenbaum	Mirpur	Dhaka
M03	Sumon	B03	Silberschatz	Uttara	Dhaka

Primary Key: `{MemberID, BookID}`

Functional Dependencies:

```
{MemberID, BookID} -> Author, Branch, BranchCity   (Full)
MemberID -> MemberName, Branch, BranchCity          (Partial)
BookID -> Author                                    (Partial)
Branch -> BranchCity                                (Transitive)
```

Status: In 1NF. Partial dependencies exist, so not in 2NF.

7.3 Step 2: Convert to 2NF

Remove partial dependencies by decomposing:

Table: Members

MemberID (PK)	MemberName	Branch	BranchCity
M01	Rahim	Central	Dhaka
M02	Karim	Mirpur	Dhaka
M03	Sumon	Uttara	Dhaka

FD: `MemberID -> MemberName, Branch, BranchCity`

Table: Books

BookID (PK)	Author
B01	Tanenbaum
B02	Navathe
B03	Silberschatz

FD: `BookID -> Author`

Table: Borrowings

MemberID (PK, FK)	BookID (PK, FK)
M01	B01
M01	B02
M02	B01
M03	B03

FD: `{MemberID, BookID}` uniquely identifies each borrowing record.

Status: All tables are in 2NF. However, in the **Members** table, a transitive dependency exists: `MemberID -> Branch -> BranchCity`.

7.4 Step 3: Convert to 3NF

Remove the transitive dependency from the **Members** table:

Table: Members (updated)

MemberID (PK)	MemberName	Branch (FK)
M01	Rahim	Central
M02	Karim	Mirpur
M03	Sumon	Uttara

FD: `MemberID -> MemberName, Branch`

Table: Branches (new)

Branch (PK)	BranchCity
Central	Dhaka
Mirpur	Dhaka
Uttara	Dhaka

FD: `Branch -> BranchCity`

Tables: Books and **Borrowings** remain unchanged from 2NF.

Final Status: All four tables (**Members**, **Branches**, **Books**, **Borrowings**) are now in 3NF. Every non-key attribute depends directly and only on the primary key of its table.

8. Summary of Normal Forms

Normal Form	Requirement	What It Eliminates
1NF	Atomic values only; no repeating groups; primary key exists	Multi-valued attributes and duplicate rows
2NF	Must be in 1NF; no partial dependencies on composite key	Partial dependencies
3NF	Must be in 2NF; no transitive dependencies on primary key	Transitive dependencies

9. Practice Problems

9.1 Problem 1

Normalize the following table to 3NF. Identify all functional dependencies at each step.

EmpID	EmpName	ProjectID	ProjectName	Hours	DeptName
E01	Nabil	P01	Website	120	IT
E01	Nabil	P02	App	80	IT
E02	Sara	P01	Website	150	IT
E03	Tanvir	P03	Research	200	R&D

Hint: Identify the composite primary key, then find partial and transitive dependencies.

9.2 Problem 2

Given the following functional dependencies for a relation `R(A, B, C, D, E)`:

```
{A, B} -> C
A -> D
D -> E
```

Decompose `R` into 3NF relations. Show the resulting tables, their primary keys, and the FDs each table satisfies.

10. Assignment

Convert the ER diagram of your lab project into a set of relational tables and apply the normalization process up to Third Normal Form (3NF). Follow the instructions below carefully:

1. **Relation Mapping:** Convert each entity and relationship from your project's ER diagram into relational tables. Clearly list each table along with its attributes and indicate the primary key and any foreign keys.

2. **Functional Dependency Identification:** For every relation obtained in Step 1, identify and document all functional dependencies. Distinguish between full, partial, and transitive dependencies where applicable. Present the dependencies using the standard notation (e.g., $A \rightarrow B$).

3. **1NF Verification:** Verify that each relation is in First Normal Form. If any table contains multi-valued or composite attributes, decompose it into 1NF. Show the table before and after the conversion and explain the changes made.

4. **2NF Conversion:** Examine each 1NF relation for partial dependencies on the primary key. If a relation has a composite primary key and contains partial dependencies, decompose it into 2NF-compliant tables. Clearly show which partial dependency was removed and how the new tables were formed.

5. **3NF Conversion:** Examine each 2NF relation for transitive dependencies. If any non-key attribute depends on another non-key attribute rather than directly on the primary key, decompose the table to achieve 3NF. Clearly show the transitive dependency chain and the resulting decomposition.

6. **Final Schema:** Present the complete set of normalized relations (all in 3NF) with their primary keys, foreign keys, and functional dependencies. Ensure that no information from the original ER diagram has been lost during the decomposition process.

Note: Each step of the conversion process must be documented with appropriate explanation. Simply listing the final tables without showing the intermediate steps and reasoning will not be accepted.

11. Conclusion

This experiment demonstrated the normalization process from unnormalized tables through 1NF, 2NF, and 3NF using functional dependencies. We saw how partial dependencies are eliminated in 2NF and transitive dependencies are eliminated in 3NF. Proper normalization leads to a well-structured database that minimizes redundancy and avoids insertion, update, and deletion anomalies. Understanding these concepts is essential for designing efficient and reliable relational databases.